并行计算期末作业

指导教师：钱莹

研究方向：计算机应用技术

姓　　名：朱君鹏

院　　系：计算机科学技术系

学　　号：52184506004

中国·上海

2019 年 6 月

快速傅氏变换与串匹配原理和实验

朱君鹏 52184506004

本文将分别介绍一种数值并行算法快速傅里叶变换 MPI 实现和一种非数值并行算法串匹配 MPI 实现。

长期以来，**快速傅氏变换**(Fast Fourier Transform)在数字信号处理、石油勘探、地震预报、医学断层诊断、编码理论、量子物理及概率论等领域中都得到了广泛的应用。各种快速傅氏变换(FFT)，成为数值代数方面最活跃的一个研究领域，而其意义远远超过了算法研究的范围，进而为诸多科技领域的研究打开了一个崭新的局面。

串匹配（String Matching）问题是计算机科学中的一个基本问题，也是复杂性理论中研究的最广泛的问题之一。它在文字编辑处理、图像处理、文献检索、自然语言识别、生物学等领域有着广泛的应用。而且，串匹配是这些应用中最耗时的核心问题，好的串匹配算法能显著地提高应用的效率。因此，研究并设计快速的串匹配算法具有重要的理论价值和实际意义。

串匹配问题实际上就是一种模式匹配问题，即在给定的文本串中找出与模式串匹配的子串的起始位置。最基本的串匹配问题是关键词匹配（Keyword Matching）。所谓关键词匹配，是指给定一个长为 n 的文本串 T[1，n]和长为 m 的模式串 P[1，m]，找出文本串 T 中与模式串所有精确匹配的子串的起始位置。串匹配问题包括精确串匹配（Perfect String Matching）、随机串匹配（Randomized String Matching）和近似串匹配（Approximate String Matching）。另外还有多维串匹配（Multidimensional String Matching）和硬件串匹配（Hardware String Matching）等。

1 快速傅里叶变换 FFT

离散傅里叶变换是 20 世纪 60 年代是计算复杂性研究的主要里程碑之一，1965 年 Cooley 和 Tukey 所研究的计算**离散傅里叶变换**(Discrete Fourier Test)的快速傅氏变换(FFT)将计算量从 O(n^2)下降至 O($n$log$n$)，推进了 FFT 更深层、更广法的研究与应用。FFT 算法有很多版本，但大体上可分为两类：迭代法和递归法，本节仅讨论迭代法。

1.1 串行 FFT 迭代算法

假定 $a[0],a[1],\cdots,a[n\text{-}1]$ 为一个有限长的输入序列，$b[0],\ b[1],\ \cdots,b[n\text{-}1]$为离散傅里叶变换的结果序列，则有：$b[k]=\sum_{m=0}^{n-1}a[k]W_n^{km}(k=0,1,2,\ldots,n-1)$，其中 $W_n=e^{\frac{2\pi i}{n}}$，实际上，上式可写成矩阵 W 和向量 a 的乘积形式：

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{n-1} \end{bmatrix} = \begin{bmatrix} w^0 & w^0 & w^0 & \cdots & w^0 \\ w^0 & w^1 & w^2 & \cdots & w^{n-1} \\ w^0 & w^2 & w^4 & \cdots & w^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w^0 & w^{(n-1)} & w^{2(n-1)} & \cdots & w^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

一般的 n 阶矩阵和 n 维向量相乘，计算时间复杂度为 n^2，但由于 W 是一种特殊矩阵，故可以降低计算量。

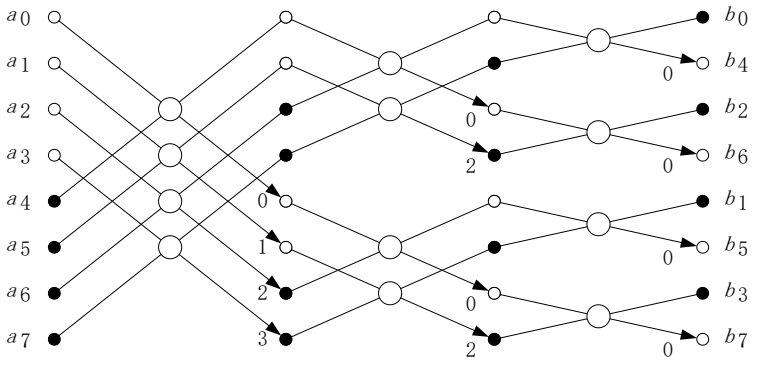

图 1　n=4 时的 FFT 蝶式变换图

显然, FFT 算法的计算复杂度为 O($n$log$n$)。

1.2 并行 FFT 算法

设 P 为处理器的个数，一种并行 FFT 实现时是将 n 维向量 a 划分成 p 个连续的 m 维子向量，这里 $m=\lceil n/p \rceil$，第 i 个子向量中下标为 $i\times m$, …, (i+1)×m-1，其元素被分配至第 i 号处理器（i=0,1, …, p-1）。由图 1 可以看到，FFT 算法由 log$n$ 步构成，依次以 $2^{\log n-1}$、$2^{\log n-2}$、…、2、1 为下标跨度做蝶式计算，我们称下标跨度为 2^h 的计算为第 h 步（h=log$n$-1, log$n$-2, …, 1, 0）。并行计算可分两阶段执行：第一阶段，第 log$n$-1 步至第 log$m$ 步，由于下标跨度 $h\geqslant m$，各处理器之间需要通信；第二阶段，第 log$m$-1 步至第 0 步各处理器之间不需要通信。具体并行算法框架描述如下：

FFT 并行算法

输入：$a=(a_0,a_1, \cdots,a_{n-1})$

输出：$b=(b_0,b_1, \cdots,b_{n-1})$

Begin

对所有处理器 my_rank(my_rank=0,…, p-1)同时执行如下的算法:

(1)**for** h=log$p$-1 **downto** 0 **do**

/* 第一阶段，第 log$n$-1 步至第 log$m$ 步各处理器之间需要通信*/

(1.1) $t=2^{i}$, $l=2^{(i+\log m)}$,$q=n/l$, $z=w^{q/2}$, $j=j+1$,$v=2^{j}$ /*开始 j=0*/

(1.2)**if** ((my_rank mod t)=(my_rank mod 2t)) **then**

/*本处理器的数据作为变换的前项数据*/

(i) tt= my_rank+p/v

(ii)接收由 tt 号处理器发来的数据块，并将接收的数据块存于 c[my_rank*m+n/v]到 c[my_rank*m+n/v+m]之中

(iii)**for** k=0 **to** m-1 **do**

$c[k]=c[k]+c[k+n/v]$

$c[k+n/v]=(c[k]-c[k+n/v])*z^{(\text{my_rank}*m+k) \bmod l}$

end for

(iv)将存于 c[my_rank*m+n/v]到 c[my_rank*m+n/v+m]之中的数据块发送到 tt 号处理器

```
            else   /*本处理器的数据作为变换的后项数据*/
              (v)将存于之中的数据块发送到 my_rank-p/v 号处理器
              (vi)接收由 my_rank-p/v 号处理器发来的数据块存于 c 中
            end if
        end for
    (2)for i=logm-1 downto 0 do   /*第二阶段，第 logm-1 步至第 0 步各处理器之间
                                    不需要通信*/
        (2.1) l=2^i   ,q=n/l , z=w^(q/2)
        (2.2)for k=0 to m-1 do
                if ((k mod l)=(k mod 2l)) then
                    c[k]=c[k]+c[k+l]
                    c[k+l]=( c[k]- c[k+l])*z^((my_rank*m+k) mod l)
                end if
            end for
        end for
End
```

由于各处理器对其局部存储器中的 m 维子向量做变换计算，计算时间为 $m\log n$；点对点通信 $2\log p$ 次，每次通信量为 m，通信时间为 $2(t_s+mt_w)\log p$，因此快速傅里叶变换的并行计算时间为 $T_p=m\log n+2(t_s+mt_w)\log p$。

2 串匹配算法

输入：分布存储的文本串和存储于 P_0 的模式串 P[1，m]

输出：所有的匹配位置

```
procedure KMP(T，P，textlen，  matched_num，match)
Begin
(1) i=1
(2) j=matched_num+1
(3) while i≤textlen do
    (3.1)while j≠0 and P[j]≠T[i] do
            j=newnext[j]
        end while
    (3.2)if j=m then
            match[i-(m-1)]=1
            j=next[m+1]
            i=i+1
        else
            j=j+1
            i=i+1
        end if
    end while
(4) maxprefixlen=j-1
End
```

附录 1 FFT 并行算法的 MPI 源程序

1. 源程序 fft.c

```
#include <stdio.h>
#include <stdlib.h>
#include <complex.h>
#include <math.h>
#include "mpi.h"

#define    MAX_PROCESSOR_NUM    12
#define    MAX_N                50
#define    PI                   3.141592653
#define    EPS                  10E-8
#define    V_TAG                99
#define    P_TAG                100
#define    Q_TAG                101
#define    R_TAG                102
#define    S_TAG                103
#define    S_TAG2               104

void evaluate(complex<double>* f, int beginPos,
int endPos, const complex<double>* x,
        complex<double> *y, int leftPos,
        int rightPos, int totalLength);

void shuffle(complex<double>* f, int beginPos,
       int endPos);

void print(const complex<double>* f,
        int fLength);

void readDoubleComplex(FILE *f,
        complex<double> &z);

int main(int argc, char *argv[])
{
        complex<double> p[MAX_N], q[MAX_N],
        s[2*MAX_N], r[2*MAX_N];
        complex<double> w[2*MAX_N];
        complex<double> temp;
        int variableNum;
        MPI_Status status;
        int rank, size;
        int i, j, k, n;
        int wLength;
        int everageLength;
        int moreLength;
        int startPos;
        int stopPos;
        FILE *fin;

MPI_Init(&argc, &argv);
MPI_Get_rank(MPI_COMM_WORLD,
        &rank);

MPI_Get_size(MPI_COMM_WORLD,
        &size);

if(rank == 0)
{
        fin = fopen("dataIn.txt", "r");
        if (fin == NULL)
        {
                puts("Not find input data file");
                puts("Please create a file
                        \"dataIn.txt\"");
                puts("<example for dataIn.txt> ");
                puts("2");
                puts("1.0    2");
                puts("2.0    -1");
                exit(-1);
        }

        readDoubleComplex(fin, variableNum);

        if ((variableNum < 1)||(variableNum >
                MAX_N))
        {
                puts("variableNum out of range!");
                exit(-1);
        }

        for(i = 0; i < variableNum; i ++)
```

```
            readDoubleComplex(fin, p[i]);
        for(i = 0; i < variableNum; i ++)
            readDoubleComplex(fin, q[i]);
        fclose(fin);

        puts("Read from data file \"dataIn.txt\"");
        printf("p(t) = ");
        print(p, variableNum);
        printf("q(t) = ");
        print(q, variableNum);

        for(i = 1; i < size; i ++)
        {
            MPI_Send(&variableNum,1,
                MPI_INT,i, V_TAG,
                MPI_COMM_WORLD);
            MPI_Send(p,variableNum,
                MPI_DOUBLE_COMPLEX,i,
                P_TAG,
                PI_COMM_WORLD);
                 MPI_Send(q,variableNum,
                MPI_DOUBLE_COMPLEX,i,
                Q_TAG,
                MPI_COMM_WORLD);
        }
}
else
{
        MPI_Recv(&variableNum,1,MPI_INT,0,
            V_TAG,MPI_COMM_WORLD,
            &status);
        MPI_Recv(p,variableNum,
            MPI_DOUBLE_COMPLEX,0,
            P_TAG, PI_COMM_WORLD,
            &status);
       MPI_Recv(q,variableNum,
            MPI_DOUBLE_COMPLEX,0,
            Q_TAG,MPI_COMM_WORLD,
            &status);
 }

wLength = 2*variableNum;
for(i = 0; i < wLength; i ++)
{
        w[i]= complex<double>
            (cos(i*2*PI/wLength),
            sin(i*2*PI/wLength));
}

everageLength = wLength / size;
moreLength = wLength % size;
startPos = moreLength + rank * everageLength;
stopPos = startPos + everageLength - 1;

if(rank == 0)
{
         startPos = 0;
         stopPos  =  moreLength+everageLength  -
            1;
}
evaluate(p, 0, variableNum - 1, w, s,
        startPos, stopPos, wLength);
evaluate(q, 0, variableNum - 1, w, r,
        startPos, stopPos, wLength);
for(i = startPos; i <= stopPos ; i ++)
        s[i] = s[i]*r[i]/(wLength*1.0);

if (rank > 0)
{
        MPI_Send((s+startPos),
            everageLength,
            MPI_DOUBLE_COMPLEX, 0,
            S_TAG, MPI_COMM_WORLD);
        MPI_Recv(s,wLength,
            MPI_DOUBLE_COMPLEX,0,
            S_ TAG2,MPI_ COMM_WORLD,
            &status);
}
else
{
        for(i = 1; i < size; i ++)
        {
            MPI_Recv((s+moreLength+i*
                everageLength),everageLength,
                MPI_DOUBLE_COMPLEX,
                i,S_TAG,
                MPI_COMM_WORLD,
                &status);
```

```
        }

        for(i = 1; i < size; i ++)
        {
                MPI_Send(s,wLength,
                        MPI_DOUBLE_COMPLEX,i,
                        S_TAG2,
                        MPI_COM M_WORLD);
        }
}

for(int i = 1; i < wLength/2; i ++)
{
        temp = w[i];
      w[i] = w[wLength - i];
      w[wLength - i] = temp;
}
evaluate(s, 0, wLength - 1, w, r, startPos,
        stopPos, wLength);

if (rank > 0)
{
      MPI_Send((r+startPos), everageLength,
              MPI_DOUBLE_COMPLEX,0,
              R_TAG,
              MPI_COMM_WORLD);
}
else
{
      for(i = 1; i < size; i ++)
      {
              MPI_Recv((r+moreLength+i*
                      everageLength),
                      everageLength,
                      MPI_DOUBLE_COMPLEX,
                      i, R_TAG,
                      MPI_COMM_WORLD,
                      &status);
      }

      puts("\nAfter FFT r(t)=p(t)q(t)");
      printf("r(t) = ");
      print(r, wLength - 1);
      puts("");
              printf("Use prossor size = %d\n",size);
      }

      MPI_Finalize();
}

void evaluate(complex<double>* f, int beginPos, int
      endPos, const complex<double>* x,
      complex<double>* y, int leftPos, int
      rightPos, int totalLength)
{
      int i;
      complex<double>
              tempX[2*MAX_N],tempY1[2*MAX_N],
              tempY2[2*MAX_N];
      int midPos = (beginPos + endPos)/2;

      if ((beginPos > endPos)||(leftPos > rightPos))
      {
              puts("Error in use Polynomial!");
              exit(-1);
      }
      else if(beginPos == endPos)
      {
              for(i = leftPos; i <= rightPos; i ++)
                      y[i] = f[beginPos];
      }
      else if(beginPos + 1 == endPos)
      {
              for(i = leftPos; i <= rightPos; i ++)
                      y[i] = f[beginPos] + f[endPos]*x[i];
      }
      else
      {
              shuffle(f, beginPos, endPos);
              for(i = leftPos; i <= rightPos; i ++)
                      tempX[i] = x[i]*x[i];
              evaluate(f, beginPos, midPos, tempX,
                      tempY1, leftPos, rightPos,totalLength);
              evaluate(f, midPos+1, endPos, tempX,
                      tempY2, leftPos, rightPos,
                      totalLength);
              for(i = leftPos; i <= rightPos; i ++)
                      y[i] = tempY1[i] + x[i]*tempY2[i];
```

```
        }
}

void shuffle(complex<double>* f, int beginPos, int
        endPos)
{
        complex<double> temp[2*MAX_N];
        int i, j;

        for(i = beginPos; i <= endPos; i ++)
                temp[i] = f[i];

        j = beginPos;
        for(i = beginPos; i <= endPos; i +=2)
        {
                f[j] = temp[i];
                j ++;
        }

        for(i = beginPos +1; i <= endPos; i += 2)
        {
                f[j] = temp[i];
                j ++;
        }
}

void print(const complex<double>* f, int fLength)
{
        bool isPrint = false;
        int i;

        if (abs(f[0].real()) > EPS)
        {
                printf("%lf", f[0].real());
                isPrint = true;
        }

        for(i = 1; i < fLength; i ++)
        {
                if (f[i].real() > EPS)
                {
                        if (isPrint) printf(" + ");
                        else isPrint = true;
                        printf("%lft^%d", f[i].real(),i);
                }
                else if (f[i].real() < - EPS)
                {
                        if(isPrint) printf(" - ");
                        else isPrint = true;
                        printf("%lft^%d", -f[i].real(),i);
                }
        }

        if (isPrint == false) printf("0");
        printf("\n");
}
```

2. 运行实例

编译： mpicc –o fft fft.cc

运行： 使用如下命令运行程序

mpirun –np 1 fft
mpirun –np 2 fft
mpirun –np 3 fft
mpirun –np 4 fft
mpirun –np 5 fft

运行结果：

Input of file "dataIn.txt"

4

1 3 3 1

0 1 2 1

Output of solution

```
Read from data file "dataIn.txt"
p(t) = 1 + 3t^1 + 3t^2 + 1t^3
q(t) = 1t^1 + 2t^2 + 1t^3

After FFT r(t)=p(t)q(t)
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 1
End of this running

Read from data file "dataIn.txt"
p(t) = 1 + 3t^1 + 3t^2 + 1t^3
q(t) = 1t^1 + 2t^2 + 1t^3

After FFT r(t)=p(t)q(t)
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 2
End of this running

Read from data file "dataIn.txt"
p(t) = 1 + 3t^1 + 3t^2 + 1t^3
q(t) = 1t^1 + 2t^2 + 1t^3

After FFT r(t)=p(t)q(t)
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 3
End of this running

Read from data file "dataIn.txt"
p(t) = 1 + 3t^1 + 3t^2 + 1t^3
q(t) = 1t^1 + 2t^2 + 1t^3

After FFT r(t)=p(t)q(t)
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 4
End of this running

Read from data file "dataIn.txt"
p(t) = 1 + 3t^1 + 3t^2 + 1t^3
q(t) = 1t^1 + 2t^2 + 1t^3
```

```
After FFT r(t)=p(t)q(t)
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 5
End of this running
```

说明：运行中可以使用参数 ProcessSize，如 mpirun －np ProcessSize fft 来运行该程序，其中 ProcessSize 是所使用的处理器个数，本实例中依次取 1、2、3、4、5 个处理器分别进行计算。

附录 2 KMP 串匹配并行算法的 MPI 源程序

1. 源程序 gen_ped.c

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <string.h>

/*根据输入参数生成模式串*/
int main(int argc,char *argv[])
{
	int strlen,pedlen,suffixlen,num,i,j;
	char *string;
	FILE *fp;

	strlen=atoi(argv[1]);
	pedlen=atoi(argv[2]);
	srand(atoi(argv[3]));

	string=(char*)malloc(strlen*sizeof(char));
	if(string==NULL)
	{
		printf("malloc error\n");
		exit(1);
	}

	for(i=0;i<pedlen;i++)
	{
		num=rand()%26;
		string[i]='a'+num;
	}

	for(j=1;j<(int)(strlen/pedlen);j++)
		strncpy(string+j*pedlen,string,pedlen);

	if((suffixlen=strlen%pedlen)!=0)
		strncpy(string+j*pedlen,string,suffixlen);

	if((fp=fopen(argv[4],"w"))!=NULL)
	{
		fprintf(fp,"%s",string);
		fclose(fp);
	}
	else
	{
		printf("file open error\n");
		exit(1);
	}

	return;
}
```

2. 源程序 kmp.c

```
#include <malloc.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <mpi.h>
#define	MAX(m,n)	(m>n?m:n)
```

```
typedef struct{
	int pedlen;
	int psuffixlen;
	int pednum;
}pntype;

/*对模式串进行周期分析，并计算相应的 new 和
	newval 值*/
void Next(char *W,int patlen,int *nextval,
	pntype *pped)
{
	int i,j,plen;
	int *next;

	if((next=(int *)malloc((patlen+1)*sizeof(int)))
		==NULL)
	{
		printf("no enough memory\n");
		exit(1);
	}

	/*计算 next 和 nextval*/
	next[0]=nextval[0]=-1;
	j=1;
	while(j<=patlen)
	{
		i=next[j-1];
		while(i!=(-1) && W[i]!=W[j-1])
			i=next[i];
		next[j]=i+1;

		if(j!=patlen)
		{
			if( W[j]!=W[i+1])
				nextval[j]=i+1;
			else
				nextval[j]=nextval[i+1];
		}
		j++;
	}

	pped->pedlen=patlen-next[patlen];
	pped->pednum=(int)(patlen/pped->pedlen);
	pped->psuffixlen=patlen%pped->pedlen;
	free(next);
}

/*改进的 KMP 算法*/
void kmp(char *T,char*W,int textlen,int patlen,
	int *nextval,pntype *pped,int prefix_flag,
	int matched_num,int *match,int *prefixlen)
{
	int i,j;

	i=matched_num;
	j=matched_num;

	while(i<textlen)
	{
		if((prefix_flag==1)&&((patlen-j)>
			(textlen-i)))
			break;
		while(j!=(-1) && W[j]!=T[i])
			j=nextval[j];
		if(j==(patlen-1))
		{
			match[i-(patlen-1)]=1;
			if(pped->pednum+pped->psuffixlen
				==1)
				j = -1 ;
			else
				j=patlen-1-pped->pedlen;
		}
		j++;
		i++;
	}
	(*prefixlen)=j;
}

/*重构模式串以及 next 函数*/
void Rebuild_info(int patlen,pntype *pped,
	int *nextval,char *W)
{
	int i;
	if (pped->pednum == 1)
		memcpy(W+pped->pedlen,W,
			pped->psuffixlen);
```

```
        else
        {
                memcpy(W+pped->pedlen,W,
                        pped->pedlen);
                for (i=3; i<=pped->pednum; i++)
                {
                        memcpy(W+(i-1)*pped->pedlen,W,
                                pped->pedlen);
                        memcpy(nextval+(i-1)*
                                pped->pedlen,
                                nextval+pped->pedlen,
                                pped->pedlen*sizeof(int));
                }

                if(pped->psuffixlen!=0)
                {
                        memcpy(W+(i-1)*pped->pedlen,W,
                                pped->psuffixlen);
                        memcpy(nextval+(i-1)*
                                pped->pedlen,nextval+
                                pped->pedlen,
                                pped->psuffixlen*sizeof(int));
                }
        }
}

/*生成文本串*/
void gen_string(int strlen,int pedlen,char *string,
        int seed)
{
        int suffixlen,num,i,j;

        srand(seed);
        for(i=0;i<pedlen;i++)
        {
                num=rand()%26;
                string[i]='a'+num;
        }
        for(j=1;j<(int)(strlen/pedlen);j++)
                strncpy(string+j*pedlen,string,pedlen);
        if((suffixlen=strlen%pedlen)!=0)
                strncpy(string+j*pedlen,string,suffixlen);
}

/*从文件读入模式串信息*/
void GetFile(char *filename,char *place, int *number)
{
        FILE *fp;
        struct stat statbuf;

        if ((fp=fopen(filename,"rb"))==NULL)
        {
                printf ("Error open file %s\n",filename);
                exit(0);
        }

        fstat(fileno(fp),&statbuf);
        if(((*place)=(char *)malloc(sizeof(char)*
                statbuf.st_size)) == NULL){
                printf ("Error alloc memory\n");
                exit(1);
        }

        if (fread((*place),1,statbuf.st_size,fp)
                !=statbuf.st_size){
                printf ("Error in reading num\n");
                exit(0);
        }
        fclose (fp);
        (*number)=statbuf.st_size;
}

/*打印运行参数信息*/
void PrintFile_info(char *filename,char *T,int id)
{
        FILE *fp;
        int i;

        if ((fp=fopen(filename,"a"))==NULL)
        {
                printf ("Error open file %s\n",filename);
                exit(0);
        }

        fprintf (fp,"The Text on node %d is %s .\n",
                id,T);
        fclose (fp);
}
```

```
/*打印匹配结果*/
void PrintFile_res(char *filename,int *t,int len,
        int init,int id)
{
        FILE *fp;
        int i;

        if ((fp=fopen(filename,"a"))==NULL)
        {
                printf ("Error open file %s\n",filename);
                exit(0);
        }

        fprintf (fp,"This is the match result on node %d
                \n",id);
        for (i=0; i<=len-1; i++)
                if(t[i]==1)
                        fprintf (fp,"(%d)    +\n",i+init);
                else
                        fprintf (fp,"(%d)    -\n",i+init);
        fclose (fp);
}

void main(int argc,char *argv[])
{
        char *T,*W;
        int     *nextval,*match;
        int     textlen,patlen,pedlen,nextlen_send;
        pntype pped;
        int     i,myid,numprocs,prefixlen,ready;
        MPI_Status    status;

        MPI_Init(&argc,&argv);
        MPI_Comm_size(MPI_COMM_WORLD,
                &numprocs);
        MPI_Comm_rank(MPI_COMM_WORLD,
                &myid);

        nextlen_send=0;
        ready=1;
        /*读如文本串长度*/
        textlen=atoi(argv[1]);
        textlen=textlen/numprocs;
        pedlen=atoi(argv[2]);
        if((T=(char *)malloc(textlen*sizeof(char)))
                ==NULL)
        {
                printf("no enough memory\n");
                exit(1);
        }
        gen_string(textlen,pedlen,T,myid);

        if(myid==0)
        {
                PrintFile_info("match_result",T,myid);
                if(numprocs>1)
                        MPI_Send(&ready,1,MPI_INT,1,0,
                                MPI_COMM_WORLD);
        }
        else
        {
                MPI_Recv(&ready,1,MPI_INT,myid-1,
                        myid-1,MPI_COMM_WORLD,
                        &status);
                PrintFile_info("match_result",T,myid);
                if(myid!=numprocs-1)
                        MPI_Send(&ready,1,MPI_INT,
                                myid+1,myid,
                                MPI_COMM_WORLD);
        }

        printf("\n");

        if((match=(int *)malloc(textlen*sizeof(int)))
                ==NULL)
        {
                printf("no enough memory\n");
                exit(1);
        }

        /*处理器 0 读入模式串，并记录运行参数*/
        if(myid==0)
        {
                printf("processor num = %d \n",
                        numprocs);
                printf("textlen = %d\n",
```

```
        textlen*numprocs);
    GetFile("pattern.dat",&W,&patlen);
    printf("patlen= %d\n",patlen);

    if((nextval=(int *)malloc(patlen*
        sizeof(int)))==NULL)
    {
        printf("no enough memory\n");
        exit(1);
    }
    /*对模式串进行分析，对应于算法 14.6
        步骤（1）*/
    Next(W,patlen,nextval,&pped);
    if(numprocs>1)
    {
        if (pped.pednum==1)
            nextlen_send = patlen;
        else
            nextlen_send = pped.pedlen*2;
    }
}

/*向各个处理器播送模式串的信息，对应于算
    法 14.6 步骤（2）*/
if(numprocs>1)
{
    MPI_Bcast(&patlen, 1, MPI_INT, 0,
        MPI_COMM_WORLD);
    if(myid!=0)
        if(((nextval=(int *)malloc(patlen*
            sizeof(int)))==NULL)
            ||((W=(char *)malloc(patlen*
            sizeof(char)))==NULL))
    {
        printf("no enough memory\n");
        exit(1);
    }
    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Bcast(&pped,3,MPI_INT,0,
        MPI_COMM_WORLD);
    MPI_Bcast(&nextlen_send,1,MPI_INT,0,
        MPI_COMM_WORLD);
    MPI_Bcast(nextval,nextlen_send,
        MPI_INT,0,
        MPI_COMM_WORLD);
    MPI_Bcast(W,pped.pedlen,
        MPI_CHAR,0,
        MPI_COMM_WORLD);
}

MPI_Barrier(MPI_COMM_WORLD);

/*调用修改过的 KMP 算法进行局部串匹配，
    对应于算法 14.6 步骤（3）*/
if(numprocs==1)
{
    kmp(T,W,textlen,patlen,nextval,
        &pped,1,0,
        match+patlen-1,&prefixlen);
}
else
{
    if(myid!=0)
    /*各个处理器分别根据部分串数据以及
        周期信息重构模式串*/
    Rebuild_info(patlen,&pped,nextval,W);
    if(myid!=numprocs-1)
        kmp(T,W,textlen,patlen,nextval,
            &pped,0,0,match+patlen-1,
            &prefixlen);
    else
        kmp(T,W,textlen,patlen,nextval,
            &pped,1,0,match+patlen-1,
            &prefixlen);

    MPI_Barrier(MPI_COMM_WORLD);

    /*各个处理器进行段间匹配，对应于算
        法 14.6 步骤（4）*/
    if(myid<numprocs-1)
        MPI_Send(&prefixlen,1,MPI_INT,
            myid+1,99,
            MPI_COMM_WORLD);
    if(myid>0)
        MPI_Recv(&prefixlen,1,MPI_INT,
            myid-1,99,
            MPI_COMM_WORLD,
            &status);
```

```
        MPI_Barrier(MPI_COMM_WORLD);
        if((myid>0)&&(prefixlen!=0))
            kmp(T-prefixlen,W,
                prefixlen+patlen-1,
                patlen,nextval,&pped,1,
                prefixlen,
                match+patlen-1-prefixlen,
                &prefixlen);

        MPI_Barrier(MPI_COMM_WORLD);
    }

    /*输出匹配结果*/
    if(myid==0)
    {
        PrintFile_res("match_result",
            match+patlen-1,textlen-patlen+1,0,
            myid);
        if(numprocs>1)
            MPI_Send(&ready,1,MPI_INT,1,0,
                MPI_COMM_WORLD);
    }
    else
    {
        MPI_Recv(&ready,1,MPI_INT,
            myid-1,myid-1,
            MPI_COMM_WORLD,&status);
        PrintFile_res("match_result",match,
            textlen,myid*textlen-patlen+1,
            myid);
        if(myid!=numprocs-1)
            MPI_Send(&ready,1,MPI_INT,
                myid+1,myid,
                MPI_COMM_WORLD);
    }
    free(T);
    free(W);
    free(nextval);
    MPI_Finalize();
}
```

3. 运行实例

编译： gcc gen_ped.c –o gen_ped
mpicc kmp.c –o kmp

运行： 首先运行 gen_ped 生成模式串，gen_ped Strlen Pedlen Seed Pattern_File。其中 Strlen 代表模式串的长度，Pedlen 代表模式串的最小周期长度，Seed 是随机函数使用的种子数，Pattern_File 是生成数据存储的文件，这里在 kmp.c 中固定指定的文件名为 pattern.dat。生成命令格式如下：

gen_ped 3 2 1 pattern.dat

之后可以使用命令 mpirun –np SIZE kmp m n 来运行该串匹配程序，其中 SIZE 是所使用的处理器个数，m 表示文本串长度，n 为文本串的周期长度。运行命令如下：

mpirun –np 3 kmp 18 2

运行结果：

存储于 pattern.dat 中的模式串为：**qmq**

存储于 match_result 中的匹配结果为：

The Text on node 0 is **asasas** .

The Text on node 1 is **qmqmqm** .

The Text on node 2 is **ypypyp .**

This is the match result on node 0

(0) -

(1) -

(2) -

(3) -

This is the match result on node 1

(4) -
(5) -
(6) +
(7) -
(8) +
(9) -
This is the match result on node 2
(10) -
(11) -
(12) -
(13) -
(14) -
(15) -
说明：运行时，令文本串长度为 18，随机产生的文本串为 asasasqmqmqmypypyp，分布在 3 个节点上；模式串长度为 3，随机产生的模式串为 qmq。最后，节点 1 上得到两个匹配位置，由+表示出来。